Gauntlets

Ready-to-run **quality gauntlets** for AI coding agents. A gauntlet is a containerized script that runs a series of checks (lint, format, tests, types) against code an agent has just written, and returns a short, structured report plus an exit code the agent (or your orchestrator) can act on.

The output is designed for **small local models** (e.g. a 30B-class model): a summary first, errors grouped and capped, and style problems auto-fixed instead of being sent back to the model.

Available gauntlets

Gauntlet	Script	Checks
Python	`Gauntlets/python_gauntlet.sh`	black, isort (auto-fix), flake8, pytest, mypy

Project layout

```
.
├── Dockerfiles/
│   └── Python/
│       └── Dockerfile          # image with all Python tools pinned
└── Gauntlets/
    ├── config/
    │   └── .flake8             # fixed config, the agent cannot chang
    ├── support/
    │   └── get_named_parameters.sh   # CLI argument parsing
    └── python_gauntlet.sh      # the gauntlet itself
```

Requirements

- Docker (20.10+)
- Nothing else: all tools live inside the image.

1. Build the image

Run from the **repository root** (the build context must be the root, because the Dockerfile copies `Gauntlets/`):

```
docker build -f Dockerfiles/Python/Dockerfile -t gauntlet-python .
```

2. Run a gauntlet

Mount the project you want to check into `/workspace` and pass the path to analyze with `-s`:

```
docker run --rm \
  -v "$PWD/my_project":/workspace \
  gauntlet-python \
  /gauntlet/python_gauntlet.sh -s /workspace/src
```

Argument	Description
`-s <path>`	Path (inside the container) of the source code to check. **Required.**

Exit codes

Code	Meaning
`0`	Gauntlet passed
`1`	At least one blocking check failed (flake8, pytest)

`docker run` propagates the script's exit code, so you can use it directly in CI or in your agent loop:

```
if ! docker run --rm -v "$PWD":/workspace gauntlet-python \
     /gauntlet/python_gauntlet.sh -s /workspace/src > report.txt; then
  echo "Gauntlet failed, feed report.txt back to the agent"
fi
```

Example output

```
=== RIEPILOGO ===
flake8: FAIL (12 errori)
pytest: ok
mypy: warning (3 errori, non bloccante)

=== flake8: errori per tipo ===
      9 F401
      3 E741

=== flake8: primi 30 ===
F401 src/app.py:3 'os' imported but unused
...

Gauntlet NON superato
```

What each check does

Check	Mode	Behavior
black	auto-fix	Reformats files in place. The agent never sees formatting errors.
isort	auto-fix	Sorts imports (`--profile black`).
flake8	blocking	Errors grouped by code, first 30 shown, with a count of the rest.
pytest	blocking	`--maxfail=3`, short tracebacks, 300 s timeout.
mypy	informational	Reported but does not change the exit code.

> ***Heads up:*** *because black and isort modify files, the project directory must be mounted* ***read-write*** *(the default with `-v`). If you want a read-only, check-only run, change `black -q` to `black --check --diff` and `isort -q` to `isort --check --diff` in the script.*

Tuning

At the top of `python_gauntlet.sh` there are a few knobs:

```
MAX_LINT=30       # flake8 detail lines
MAX_MYPY=20       # mypy detail lines
MAX_PYTEST=80     # pytest output lines
TEST_TIMEOUT=300  # seconds
```

Lower them if your model loses track with long outputs; raise them if it needs more iterations to see all the problems.

The flake8 rules live in `Gauntlets/config/.flake8` and are baked into the image, so the project being checked cannot override them.

Project dependencies (pytest / mypy)

`pytest` and `mypy` run with the **container's Python**. If your project has third-party dependencies, they must be installed in the container before the gauntlet runs. Two common options:

```
# Option A: install at run time (requires network)
docker run --rm -v "$PWD":/workspace gauntlet-python bash -c \
  "pip install -q -r /workspace/requirements.txt && \
   /gauntlet/python_gauntlet.sh -s /workspace/src"

# Option B: build a derived image with your dependencies
#   FROM gauntlet-python
#   COPY requirements.txt .
#   RUN pip install --no-cache-dir -r requirements.txt
```

3. Export and share the image

As a tar file (offline / air-gapped machines)

```
# Export
docker save gauntlet-python | gzip > gauntlet-python.tar.gz

# Import on another machine
docker load < gauntlet-python.tar.gz
```

Through a registry (recommended)

Example with GitHub Container Registry:

```
echo "$GITHUB_TOKEN" | docker login ghcr.io -u <your-user> --password-st

docker tag gauntlet-python ghcr.io/<your-user>/gauntlet-python:latest
docker push ghcr.io/<your-user>/gauntlet-python:latest
```

Anyone can then skip the build step:

```
docker pull ghcr.io/<your-user>/gauntlet-python:latest
```

Just share the Dockerfile

Since the image is built from this repository, cloning it and running the build command from step 1 is enough to get an identical image: tool versions are pinned in the Dockerfile.

Adding a new gauntlet

1. Create `Gauntlets/<name>_gauntlet.sh`, reusing `support/get_named_parameters.sh`.
2. Add the tools it needs (pinned) to a Dockerfile under `Dockerfiles/<Language>/`.
3. Follow the same output contract: **summary first, details capped, exit code `0`/`1`.**

Design principles

- **Pinned versions** for reproducible results.
- **Fixed config inside the image**, so an agent cannot “cheat” by editing lint rules.
- **Auto-fix what can be fixed**, send the model only what needs reasoning.
- **Capped, grouped output** to protect the model's context window.
- **Disposable containers**: run with `--rm`, and use `--network none` when you don't need to install anything.

License

Add your license here.